1. Allgemeine Grundkenntnisse

1. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Top-Down-Entwurf besteht darin, Details erst grob, dann feingliedrig zu sortieren.
   - (B) Algorithmen sind endliche Vorschriften, die Rechenabläufe schrittweise präzise beschreiben.
   - (C) Compiler übersetzen Quellcode erst in Assembler, dann in Unix-Code.
   - (D) Das Java-Programm ist auf dem System lauffähig, wenn darauf der entsprechende Compiler installiert ist.
2. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Bei der Entwicklung eines Programms muss nach jedem Compilieren editiert werden.
   - (B) Interaktive Programme können Daten von der Standard-Eingabe erhalten.
   - (C) Am CMD-Flag ist erkennbar, ob die Speicherzelle eine Zahl oder einen Befehl enthält.
   - (D) Eine *Pipe* funktioniert im wesentlichen wie eine Queue.
3. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Durch Eingabeumleitung kann man gespeicherte Daten an ein interaktives Kommando leiten.
   - (B) Informationsverarbeitung kann man zerlegen in die Schritte Codierung, Datenverarbeitung und Decodierung.
   - (C) Mit 7 Hexadezimalziffern kann man mehr als 1 Milliarde Werte darstellen.
   - (D) Der Bruch 4/5=0.8 kann exakt als `double`-Wert dargestellt werden.
4. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Ganzzahlarithmetik ist normalerweise schneller als Gleitkomma-Arithmetik.
   - (B) Strukturierte Programmierung ist die Programmierung mit strukturierten Daten.
   - (C) Rekursion benötigt mehr Speicher auf dem Stack als Iteration.
   - (D) Die JVM interpretiert jeden Bytecodebefehl und prüft ihn auf syntaktische Korrektheit.

2. Einige Java-Grundkenntnisse

1. Ordnen Sie Begriffe zeitlich (0.25P für jede korrekte Position)
2. Füllen Sie die Lücken, so dass das Programm mit zwei beliebigen Zahlwerten A und B als Kommandozeilenparametern aufgerufen Folgendes leistet: Es wird die Summe der ganzen Zahlen ausgeben, die innerhalb des *abgeschlossenen* Intervalls [A,B] liegen. (0.25P für jede korrekte Angabe)

```
public class Intervallsumme {
    public static void main(String[] args) {
        ,
            min = (args[0]),
            max = (args[1]);
        int ;
        while ( ) {
            ;
            ;
        }

    }
}
```

3. Wieviele Bytes beansprucht eine Variable?
    - `int`:
    - `char`:
    - `float`:
    - `boolean`:

    (0.25P für jede richtige Antwort)

3. Grundlegende Datentypen

1. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Das Leerzeichen ist ein druckbares Zeichen.
   - (B) iso8859-1 ist eine Teilmenge von ASCII.
   - (C) `'C'++` liefert das gleiche Ergebnis wie `'C'+1`.
   - (D) `"C" + 1` ist *kein* erlaubter Java-Ausdruck.
2. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) `int` ist ein Ganzzahldatentyp, `char` nicht.
   - (B) `int x = Integer.MAX_VALUE + 1;` ist eine erlaubte Anweisung.
   - (C) Der Wert von `( 3 < 7 ) && ((17 + 4) % (12 - 5)) != 0)` ist `false`.
   - (D) Der Wert von `( 3 + 1/3 == 3 - 1/3)` ist `false`.
3. Wählen Sie die zutreffenden Antworten aus:
   - (A) `String s = "Hallo"; System.out.println(s.length);`

     Ausgabe?
   - (B) `System.out.println(10e-1f);`

     Ausgabe?
   - (C) `2 * (short) 0xCafe`

     Typ des Ausdrucks?
   - (D) `double y = Double.MIN_VALUE; System.out.println(y/2);`

     Ausgabe?
   - (E) `int x = Integer.MAX_VALUE, y = Integer.MAX_VALUE++, z = x+y;`

     Wert von `z`?
   - (F) `System.out.println( (double) (1/2));`

     Ausgabe?
4. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) `int[] z = {1}; System.out.println(z);` produziert die Ausgabe 1.
   - (B) `length` ist ein `private`-Attribut von Feldern.
   - (C) Durch `int[] a = {1, 2, 3}, b = new int[];` werden zwei neue Felder angelegt.
   - (D) Durch `int[][][][] x = new char[32][32][32][16];` wird etwa 1Mb Heap-Speicher reserviert.

4. Integer-Arithmetik

Sei `d` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
1:  System.out.println(-d);     //
2:  System.out.println(d-1);    //
3:  System.out.println(1+2*d);  //
4:  System.out.println(-(2*d)); //
```

Achten Sie darauf, dass Ihre Antworten NUR Dezimalziffern enthalten (sowie bei Bedarf Vorzeichen)!

5. Iteration in Feldern

```
int[] p = { 1, 4, 0, 3, 6, 2, 7, 5 };
int[] c = { 0, 0, 0, 0, 0, 0, 0, 0 };
int[] q = { 0, 0, 0, 0, 0, 0, 0, 0 };
for (int i = 0; i < 8; i++)
    c[p[i]] = c[p[i]] + 1;
for (int i = 0; i < 8; i++)
    q[p[i]] = i;
```

Tragen Sie die Werte der Feldkomponenten *nach* Ausführung des Java-Fragments ein:

Index	0	1	2	3	4	5	6	7
c								
q								

6. Felder: Fehler oder...?

Erzeugt untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- eine Ausgabe? Falls ja, so tragen Sie Ausgabe ein, ansonsten einen Bindestrich (-):

```
1: class FehlerOderKeiner {
2:     public static void main(String[] args) {
3:         int[] a = {2,3,5,7,11}; int[] b = { a[2], a[2]};
4:         b = a;
5:         b[1] = b[0];
6:         System.out.println(a[0] != a[1]);
7:     }
8: }
```

7. Rekursion

Betrachten Sie folgende Klasse:

```
 1: class Myst {
 2:     static void ery(int n) {
 3:         if (n == 0)
 4:             return;
 5:         ery( n-1);
 6:         A();
 7:         yre( n-1);
 8:     }
 9:     static void yre(int n) {
10:         if (n == 0)
11:             return;
12:         ery( n-1);
13:         B();
14:         yre( n-1);
15:     }
16:     static void A() {
17:         System.out.print("A");
18:     }
19:     static void B() {
20:         System.out.print("B");
21:     }
22:     public static void main(String[] args) {
23:         int n = Integer.parseInt(args[0]);
24:         ery(n);
25:         System.out.println();
26:     }
27: }
```

1. Geben Sie die ersten 8 Zeichen der Ausgabe zum Aufruf `java Myst 4` an:
2. Aus wievielen Zeichen besteht der Ausgabestring beim Aufruf `java Myst 7` (ohne den abschließenden Zeilenwechsel durch Zeile 25)?
3. Wird Zeile 14 durch `yre( n-2);` ersetzt, so führt beispielsweise der Aufruf `java Myst 4` zu einem Fehler.
   Es genügt, ein einzelnes Zeichen in einer *anderen* Zeile zu ändern, um diesen Fehler nicht mehr hervorzurufen. Um welche Zeile handelt es sich?
4. Werde Zeile 12 durch `ery( n-2);` und zusätzlich Zeile 14 durch `yre( n-2)` ersetzt.
   Geben Sie das maximale Argument `n` an, bei dem der Aufruf `java Myst n` *nicht* zu einem Fehler führt:

8. Stack und Queue in Aktion

1. Auf einem Stack werden *in gemischter Reihenfolge* 10 Push- und 10 Pop-Operationen ausgeführt. Es ist bekannt, dass die Push-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?

   Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 2P für die richtige Kombination/0P für jede andere).

   - (A) 1 2 3 4 5 6 9 8 7 0
   - (B) 0 4 6 5 3 8 1 7 2 9

   - (C) 1 4 7 9 8 6 5 3 0 2
   - (D) 2 1 4 3 6 5 8 7 9 0

2. Auf einer Queue werden *in gemischter Reihenfolge* 10 Enqueue- und 10 Dequeue-Operationen ausgeführt. Es ist bekannt, dass die Enqueue-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Dequeue der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?

   Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 1P für die richtige Kombination/0P für jede andere).

   - (E) 4 6 8 7 5 3 2 9 0 1
   - (F) 2 5 6 7 4 8 9 3 1 0

   - (G) 0 1 2 3 4 5 6 7 8 9
   - (H) 4 3 2 1 0 5 6 7 8 9

9. Queue aus zwei Stacks bauen

Mit zwei Stacks kann man eine Queue implementieren:

- `enqueue(...)` entspricht Einspeichern in `inbox`
- `dequeue()` entspricht Ausspeichern aus `outbox`.

Unter Verwendung dieser API

`public class Stack`	
`boolean empty()`	testet, ob dieser Stack leer ist
`void push(String s)`	speichert `s` in diesem Stack
`String pop()`	liefert zuletzt gespeicherten String aus diesem Stack und entfernt ihn

sähe die entsprechende Klasse z.B. so aus:

```
public class ZSQueue { // Zwei-Stack-Queue
    private Stack inbox, outbox;
    public ZSQueue() { ... }
    public boolean empty() { ... }        // Leerheitstest
    public void enqueue(String s) { ... } // einspeichern
    public String dequeue() {             // ausspeichern und liefern
        // Rumpf (Ihre Aufgabe)
    }
}
```

Stellen Sie die Zeilen für den Rumpf der Methode `dequeue()` richtiger Reihenfolge zusammen:

- ist die Queue leer, ist Ausspeichern nicht möglich!
- teste ob `outbox` leer ist und fülle in diesem Fall den kompletten Inhalt von `inbox` um in `outbox`, dann ein Element aus `outbox` ausspeichern und liefern.

10. Vermischtes zu Klassentypen

1. Objekte verstehen:

```
public class Spieler {
    private int score;
    public Spieler( int s) {
        this.score = s;
    }
}
```

Zwei `Spieler`-Objekte werden instanziiert:

```
Spieler a = new Spieler(50), b = new Spieler(50);
```

Welchen Wert hat nun der Ausdruck a == b?

2. Fremden Code deuten:

```
1:  public class FirstBpp extends hepno.users.Frame {
2:      private hepno.users.Wring wrags;
3:      public FirstBpp() {
4:          wrags = new hepno.users.Wring();
5:      }
6:      public static void main(String[] args) {
7:          FirstBpp app = new FirstBpp();
8:      }
9:  }
```

Was bewirkt Zeile 2?:

3. Statische Methoden und Instanzmethoden:

```
public class Klasse {
    private final int zahl = 10;
    double a() {
        System.out.println(zahl);
        return c();
    }
    void b() {
        System.out.println(this);
    }
    double c() {
        double r = Math.random();
        System.out.println(r);
        return r;
    }
    void d() {
        a();
        a();
    }
    int e() {
        return zahl;
    }
}
```

Welche der obigen Methoden ist ihrer Natur nach eher eine statische Methode (und sollte daher mit `static` versehen werden)?

4. Fehlersuche:

```
public class Test {
    private int testCount;
    public static int getCount(){
        return testCount;
    }
    public Test(){
        testCount++;
    }
}
```

Obiger Code verursacht einen Compilezeitfehler. Was ist das Problem?

5. Tracing:
   Hinweis: Beachten Sie, dass man über Objektreferenzen auch auf Klassenvariablen zugreifen kann.

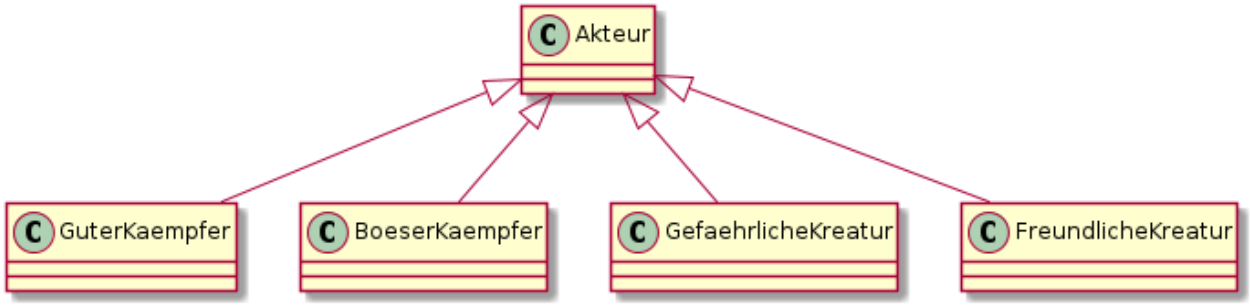
```
public class Tset {
    static int tsetCount = 0;
    public Tset(){
        tsetCount++;
    }
    public static void main(String [] args){
        Tset t1 = new Tset();
        System.out.print(t1.tsetCount + ", ");
        Tset t2 = new Tset();
        System.out.print(t1.tsetCount + " "+ t2.tsetCount + ", ");
        Tset t3 = new Tset();
        System.out.print(t1.tsetCount+ " "+ t2.tsetCount+ " "+ t3.tsetCount);
        System.out.println();
    }
}
```

   Welche Ausgabe produziert der Code?

6. Begriffsverständnis: Wählen Sie aus dem Menü die einzige Aussage aus, die NICHT korrekt ist.

11. OOP-Entwurf

In einer Spiele-Software gibt es folgende Hierarchie von Java-Klassen, die die verschiedenen Sorten von Akteuren modelliert:



Die (hier nicht gezeigte) Klasse `Spiel` benutzt die zugreifbaren Methoden obiger Klassen, um den Ablauf zu gestalten.

Wie wird folgende Zusatzanforderung korrekt modelliert?

Alle Kaempfer-Objekte aber keine Kreatur-Objekte sollen mit einer Methode benutzeWaffe() ausgestattet werden.

- `benutzeWaffe()` in `Akteur` implementieren, in `GefaerlicheKreatur` und `FreundlicheKreatur` mit `private` kennzeichnen.
- Interface `Kaempfer` mit `benutzeWaffe()` definieren und durch `GuterKaempfer` und `BoeserKaempfer` implementieren lassen.
- Die Methode `benutzeWaffe()` nur in `GuterKaempfer` und in `BoeserKaempfer` wie benötigt implementieren.
- Weitere Klasse `Kaempfer` schreiben, die `benutzeWaffe()` implementiert, `GuterKaempfer`, `BoeserKaempfer` davon erben lassen.
- `benutzeWaffe()` mit leerem Rumpf in `Akteur` implementieren, in `GuterKaempfer` und `BoeserKaempfer` geeignet überschreiben.

12. OOP-Verständnisfragen

Sei `p` eine Referenzvariable vom Typ `X`. Was kann alles der dynamische Typ von `p` sein? Wählen Sie aus den beiden Blöcken unten jeweils die zutreffenden Antworten aus (jeweils 1.5P für die richtige Kombination/0P für jede andere).

- (A) eine Erweiterungsklasse von `X` (falls `X` eine Klasse ist)
- (B) ein elementarer Datentyp (falls `X` die zugehörige Wrapper-Klasse ist)
- (C) eine Klasse, die `X` implementiert (falls `X` ein Interface ist)

- (D) eine Klasse, von der `X` direkt oder indirekt erbt (falls `X` eine Klasse ist)
- (E) ein Interface, das `X` als abstrakte Klasse erweitert (falls `X` eine Klasse ist)
- (F) eine Erweiterungsklasse einer Klasse, die `X` implementiert (falls `X` ein Interface ist)

13. Analyse von Algorithmen

1. Betrachten Sie folgenden Ausschnitt aus einem größeren Programm:

```
1: for (int i = 0; i < N; i++)
2:     for (int j = i+1; j < N; j++)
3:         for (int k = j+1; k < N; k++)
4:             verarbeite(daten);
```

   Die Ausführungszahl $V(N)$ von Zeile 4 bei Problemgröße N soll untersucht werden. Welcher der folgenden Ausdrücke ist am besten für die weitere asymptotische Analyse geeignet?

2. Eine Implementation wird zahlreichen Verdopplungstests unterzogen, mit zufällig generierten Eingabedaten. Die Ergebnisse — alle sehr ähnlich — sehen typischerweise so aus:

Problemgröße N	1024	2048	4096	8192	16384	32768	65536	131072
Zeit (s)	0.00	0.01	0.01	0.02	0.03	0.06	0.14	0.30

262144	524288	1048576	2097152	4194304	8388608
0.65	1.24	2.46	4.98	10.02	19.94

   Außerdem kann man nachweisen, dass die Ausführungszahl der Kernoperationen bei spezieller Struktur der Eingabe linear mit der Problemgröße N wächst.

   Welche dieser Hypothesen über das Laufzeitverhalten erscheint dann am plausibelsten?

3. Welche dieser Aussagen trifft NICHT zu?

14. Eine Bibliothek für Permutationen

Definition

1. Wir nennen ein `int`-Feld `p` der Länge `n` eine *Permutation*, wenn `p[0]`, …, `p[n-1]` im Bereich $0, 1, \ldots, n-1$ liegen und jeder Wert genau einmal vorkommt.
2. Die Permutation `q` heißt *Inverse* der Permutation `p`, wenn für alle `i` der Wert von `q[p[i]]` gleich `i` ist.

Aufgabe

Schreiben Sie eine Klasse `Permutation` mit den *statischen* Methoden `check(..)` und `invers(..)`, die aus jeder Clientklasse aufgerufen werden können und Folgendes leisten:

- `check(p)` überprüft, ob `p` eine Permutation ist und liefert das Ergebnis (**Hinweis**: z.B. zuerst in einer Schleife prüfen, ob die Bereichsbedingung erfüllt ist, dann mit zweiter Schleife in einem Hilfsfeld `c` die Vorkommen der Werte `p[0]`, …, `p[n-1]` erfassen)
- `invers(p)` ermittelt und liefert die Inverse von `p`. (Dabei ist bereits gesichert, dass `p` wirklich eine Permutation ist - nicht separat prüfen! **Hinweis**: Feld anlegen und in Schleife die Komponenten so setzen, wie es Definition 2. vorschreibt.)

15. Eine einfache Klasse

Programmieren Sie eine einfache Klasse `Auto` mit der Angaben zu Autos abgespeichert werden können. Jedes Auto zeichnet sich durch seinen Typ aus (z.B. "Mercedes") und durch seinen Preis (z.B. 30000). Die folgenden Operationen sollen durch Methoden/Konstruktoren ermöglicht werden (*wenn nicht anders angegeben*: aufrufbar aus beliebigen Clientklassen):

- Erzeugen einer `Auto`-Instanz unter Angabe des Typs, der Typ soll nach Erzeugen des Objekts nicht mehr änderbar sein
- Erzeugen einer `Auto`-Instanz unter Angabe des Typs und des Preises,
- Setzen des Preises einer Instanz (*nur von `Auto`-Methoden aus*),
- Abfragen des Preises einer Instanz
- Abfragen der Anzahl der erzeugten Instanzen und der Summe der Preise

VERWENDEN SIE FÜR DIE DATEN BITTE DIESE BEZEICHNER:
`typ`, `preis`, `anzahl`, `gesamt`